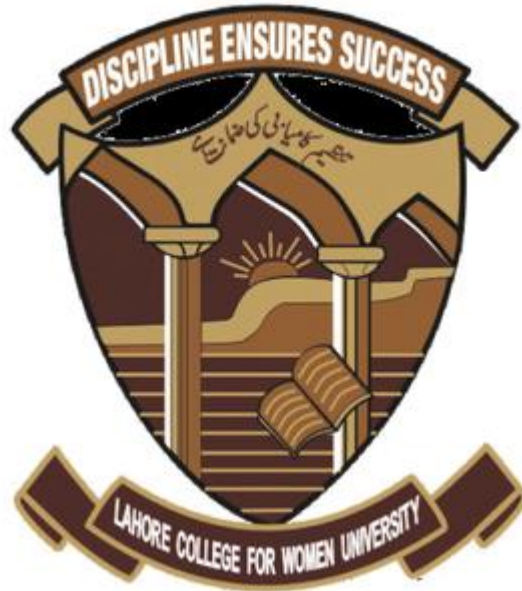


OOAD PROJECT



Submitted By:

Fabiya Noor(23251101023)

Fatima Idrees(23251101027)

Fatima Nadeem(23251101028)

Zainab Shahid(23251101118)

Submitted To:

Dr Komal Bashir

**DEPARTMENT OF COMPUTER SCIENCE
LAHORE COLLEGE FOR WOMEN
UNIVERSITY**

Simplified Git Version Control System (SGVCS)

1. Executive Summary

The Simplified Git Version Control System (SGVCS) is a foundational software engineering project designed to replicate the core mechanisms of modern version control. While standard enterprise systems are highly complex, SGVCS focuses on the essential OOAD principles—encapsulation, state management, and modular system design. The system provides a command-line interface for users to track changes, manage parallel development through branches, and synchronize repositories across distributed environments.

2. System Overview

2.1 Problem Statement

In software development, the ability to maintain version history and collaborate safely is paramount. Without version control, developers risk losing work, overwriting changes, and failing to manage experimental code paths.

2.2 Project Goals

- **Version Tracking:** Implementing a mechanism to capture "snapshots" of file states.
- **Concurrency:** Enabling safe concurrent development through branching and merging.
- **Distributed Collaboration:** Facilitating synchronization with remote repository servers.

2.3 System Actors

- **Developer (Primary Actor):** An individual contributor responsible for managing the local repository, performing commits, and synchronizing with remote servers.
- **Administrator (Secondary Actor):** A user with elevated privileges responsible for configuring remote repository access rights and server-side permissions.

3. Functional Requirements (Use Case Modeling)

The following use cases define the system's behavior from the perspective of the users.

- **Initialize Repository**

UC-01: Initialize Repository

A user creates a new project folder and initializes it as a Git-tracked directory.

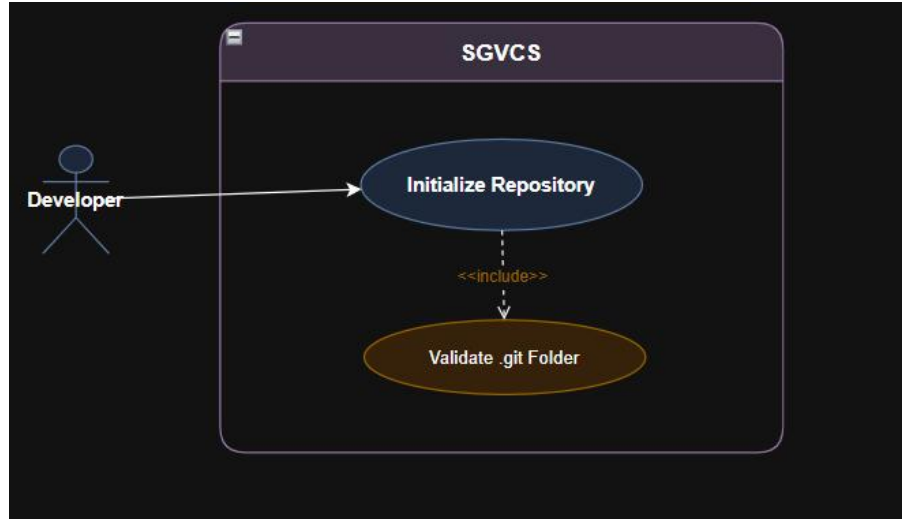


Figure 1.1: Use Case Diagram – Initialize Repository

- **Local File Operations**

UC-02: Add Files

The user moves changes from the working directory to the staging area.

UC-03: Commit Changes

The user creates a permanent snapshot of staged files with a unique identifier and associated metadata.

UC-04: View Status

The user checks the repository status to identify modified, staged, and untracked files.

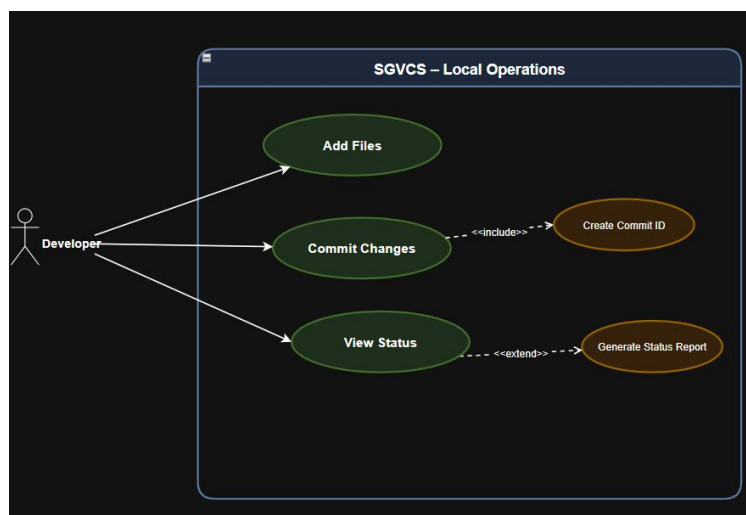


Figure 1.2: Use Case Diagram – Local File Operations

- **Branching and Merging**

UC-05: Create Branch

The user creates a new branch pointer from the current commit to support isolated feature development.

UC-06: Switch Branch

The user updates the local workspace to display the content of a selected branch.

UC-07: Merge Branches

The user combines changes from one branch into the current working branch.

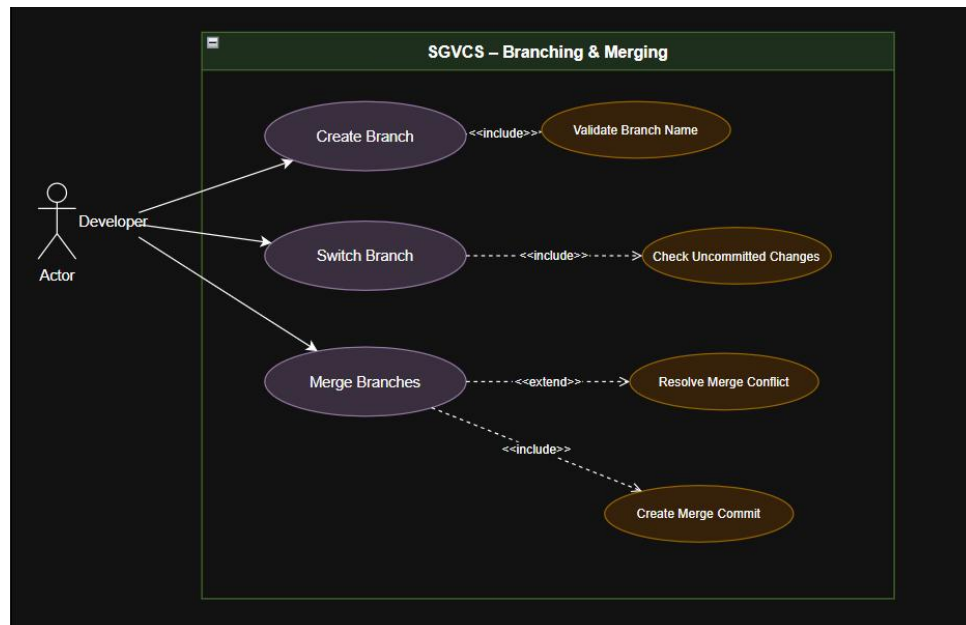


Figure 1.3: Use Case Diagram – Branching and Merging

- **Network Operations**

UC-08: Clone Repository

The user downloads a complete remote repository to a local machine.

UC-09: Push to Remote

The user uploads local commit history and changes to a remote repository.

UC-10: Pull from Remote

The user retrieves and integrates updates from a remote repository into the local workspace.

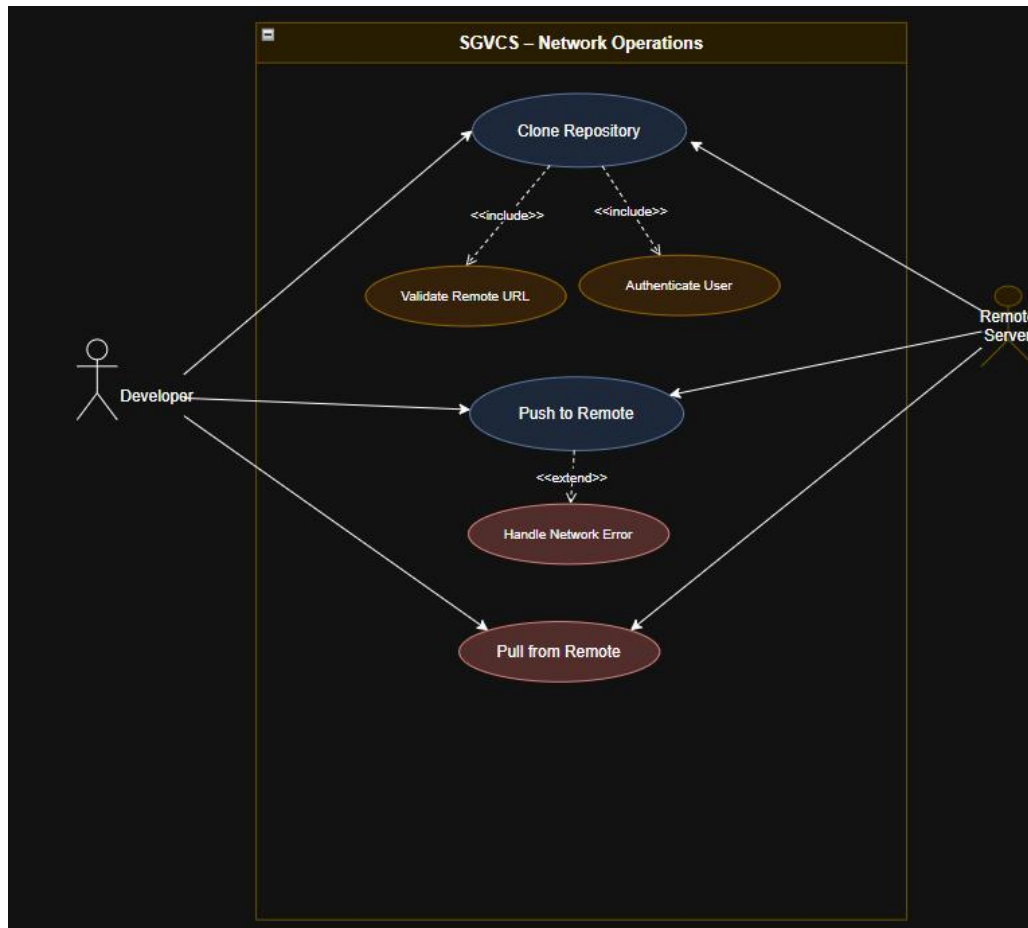


Figure 1.4: Use Case Diagram – Network Operations

- **Complete System Overview**

The following diagram provides a complete overview of all use cases, representing every functional requirement of the SGVCS system.

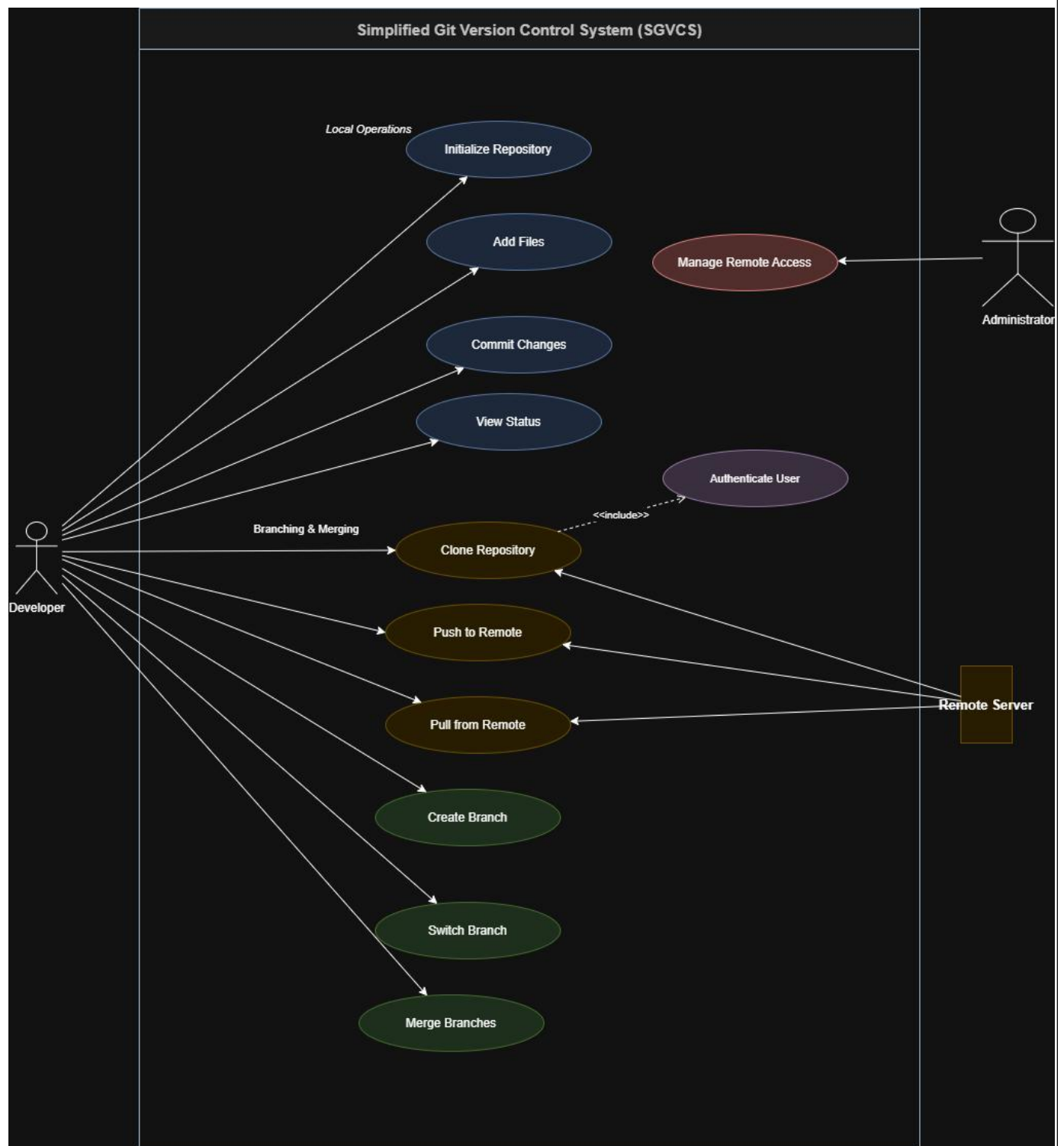


Figure 1.5: Complete Use Case Diagram – All Operations

4. Behavioral Analysis (Activity Modeling)

Activity diagrams represent the logic flow, including decision points and error handling. These models translate the functional requirements into actionable procedural steps. The following diagram provides a system-level overview of the SGVCS workflow, followed by granular diagrams for specific operations.

4.0 System-Level Workflow

This diagram provides a comprehensive representation of the SGVCS process lifecycle. It captures the global flow of the system, starting from the initial repository setup and moving through the iterative development loop.

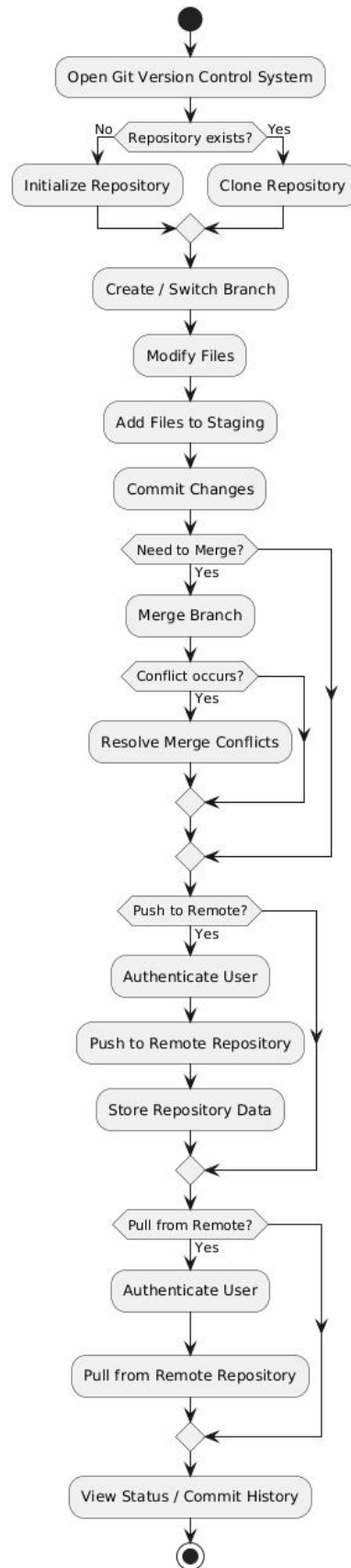
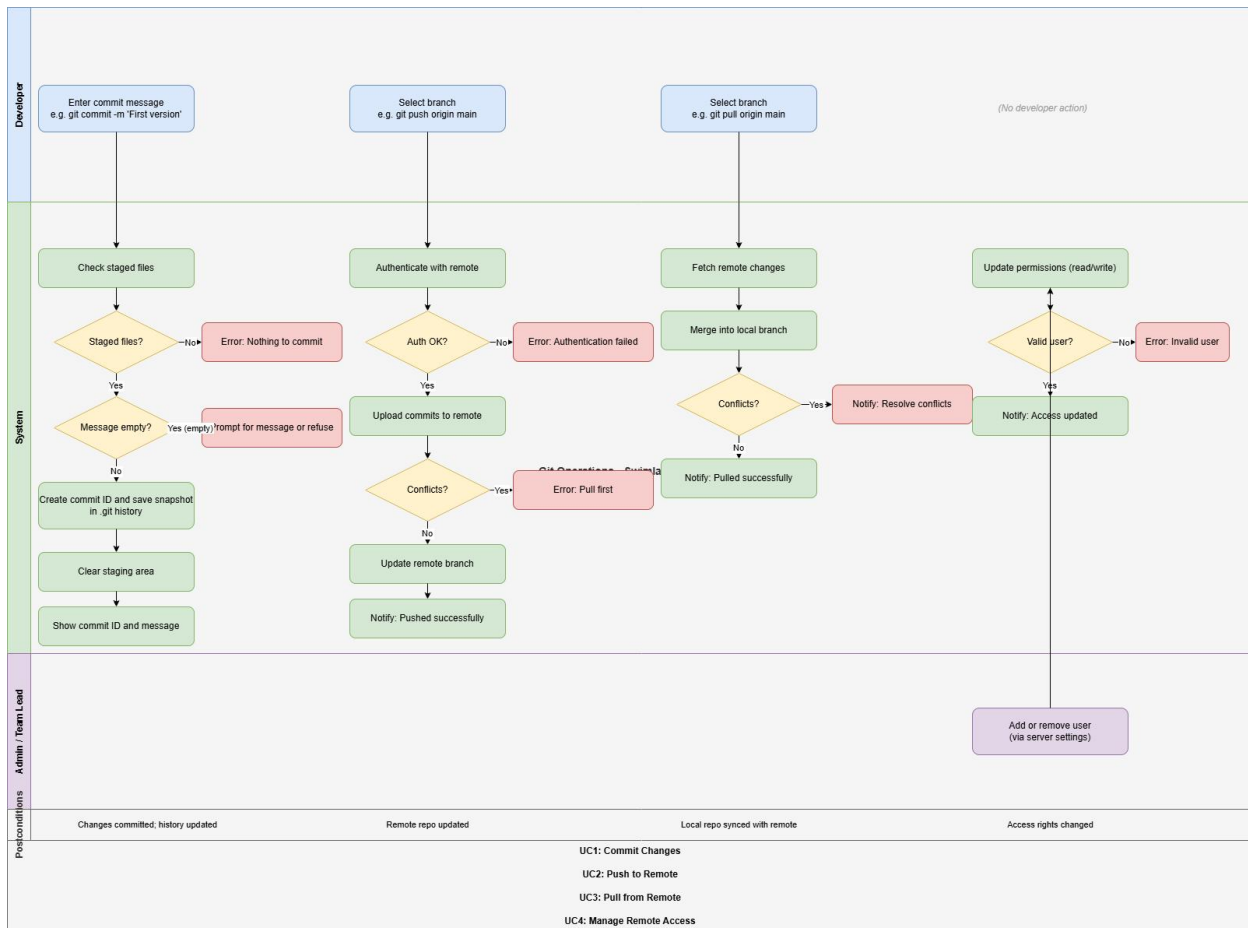


Figure 4.0: System-Level Workflow Activity Diagram

This diagram provides a high-level representation of the SGVCS process lifecycle. It captures the end-to-end flow of the system, starting from the initial setup (Initialization or Cloning) and moving through the core development loop (Branching, Modifying, Committing). The flow highlights the critical decision-making gateways, such as repository state (New vs. Existing), conflict resolution, and network synchronization (Push/Pull). It emphasizes the circular nature of the version control workflow, ensuring that the system state is always transparent to the developer.

Swimlane Diagram

A swimlane diagram is a type of flowchart that visually breaks down a process while showing who (which role, department, or system) is responsible for each task. It gets its name from the parallel horizontal or vertical "lanes" that separate different participants, similar to lanes in a swimming pool.



4.1 Local Repository Operations

- **Initialization of Repository:** This diagram details the lifecycle of repository creation and loading. It illustrates the dual-path nature of the initialization process: creating a new repository or loading an existing one. The logic incorporates essential error handling, such as checking for the presence of the `.git` metadata folder, ensuring that the system maintains filesystem integrity before proceeding with any version control operations.



Figure 2.1 Initialization of Repository

- **Staging Logic:** This activity model formalizes the transition of files from the working directory to the staging area. It demonstrates input validation by checking for the existence of files and identifying whether there are pending changes. By modeling the decision node for "Single vs. All" file addition, the diagram captures the system's flexibility in allowing targeted updates versus comprehensive staging, providing a robust workflow for the developer.

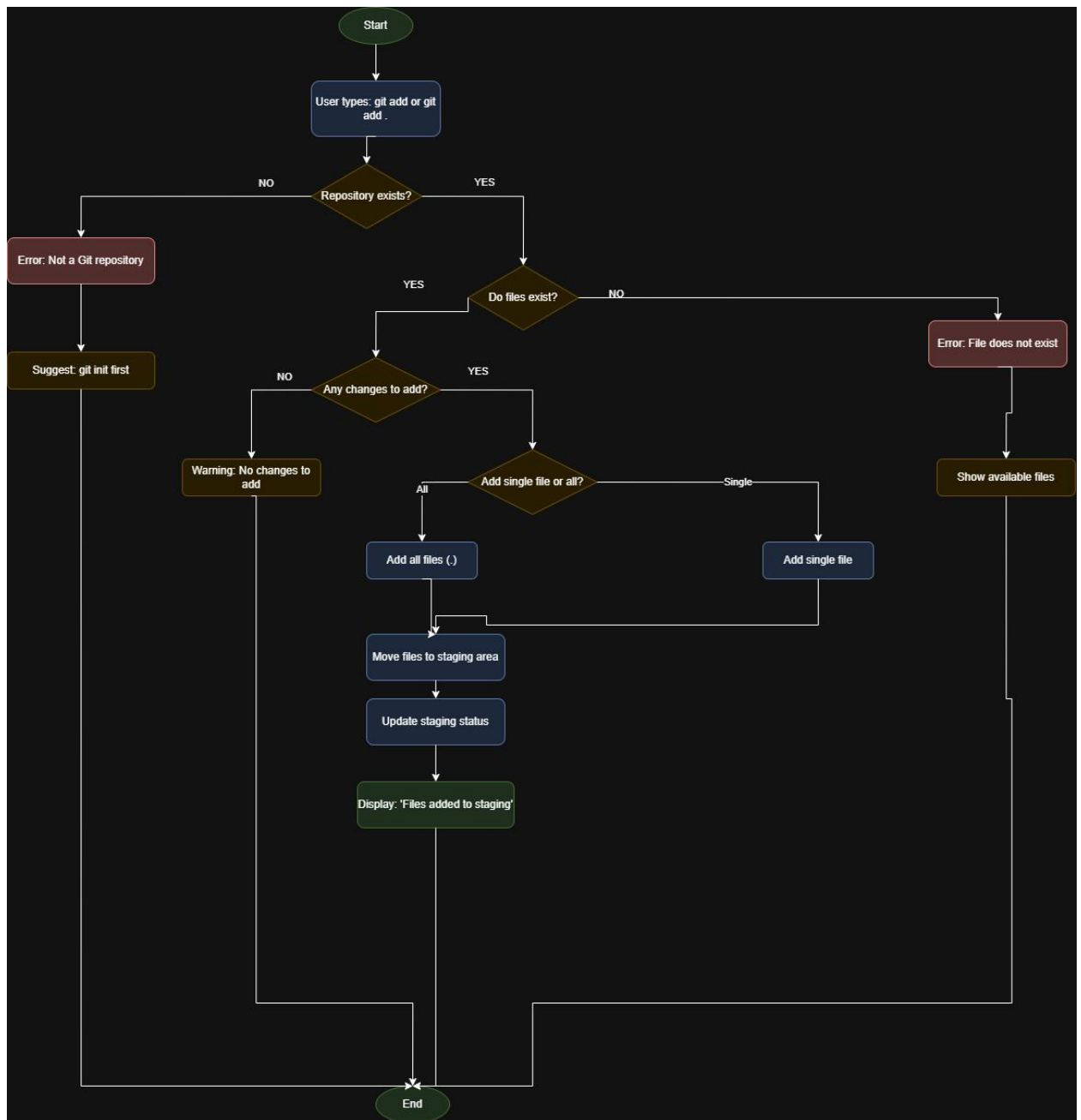


Figure 2.2: Staging Logic (Upload to Stage)

- **Status Monitoring:** The View Status diagram represents the system's diagnostic capability. It outlines the sequence of scanning the working directory, the staging area, and the commit history. The model highlights the decision-making process required to determine the state of the repository—specifically distinguishing between a "clean" directory and one containing modified or untracked files—to provide real-time feedback to the user.

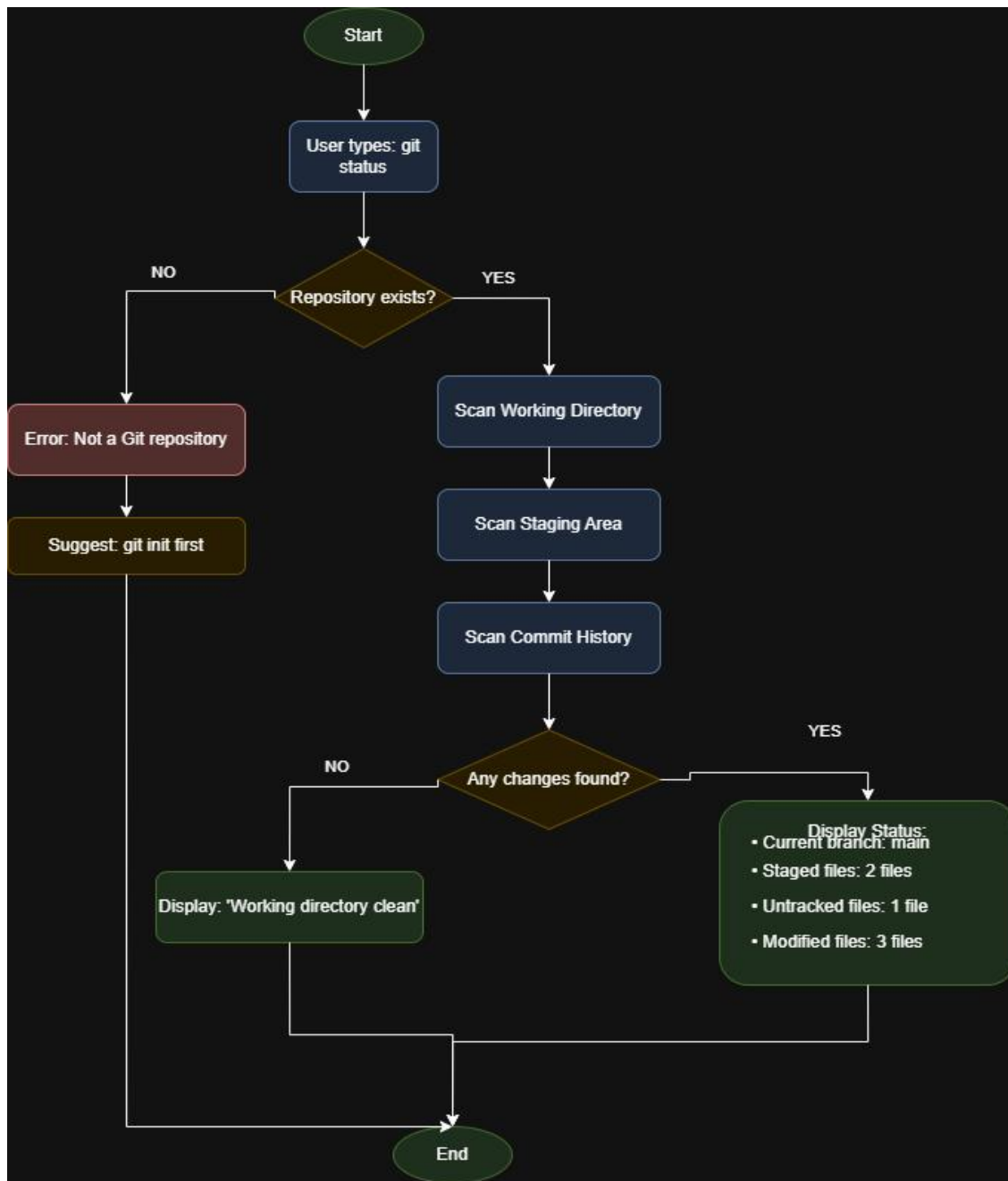


Figure 2.3: Status Monitoring (View Status)

4.2 Branching and Integration

- **Branch Creation:** This diagram focuses on the logic required to instantiate a new branch pointer. It details the process of validating the uniqueness of the branch name and the creation of a new branch pointer at the current HEAD commit. It includes a post-creation decision node that allows the user to switch to the newly created branch immediately, streamlining the development workflow.

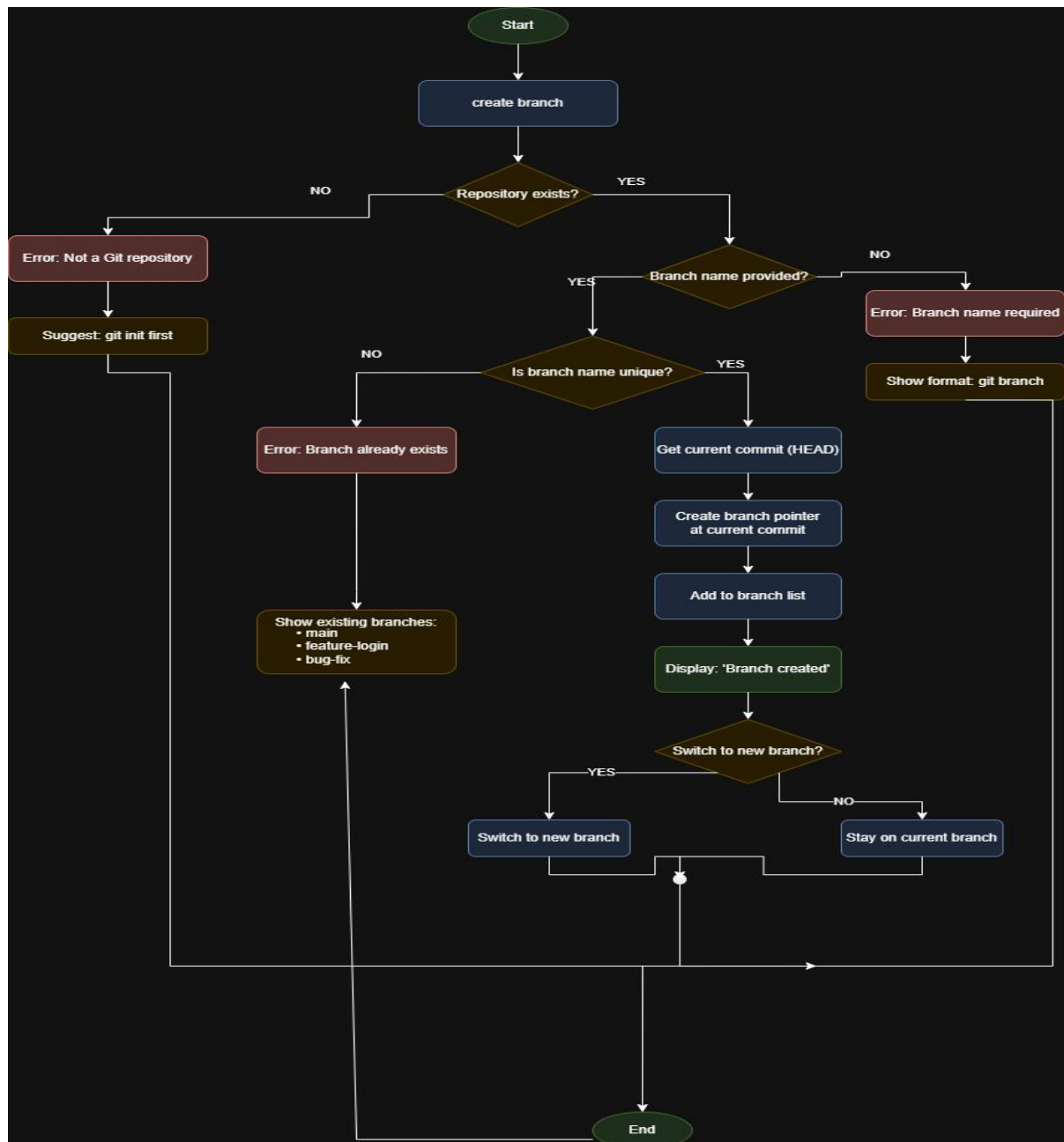


Figure 2.4: Branch Creation

- **Switching Branches:** The switching branch logic is designed to safeguard work-in-progress. Before updating the working directory to a new branch, the activity diagram models a critical decision node that identifies uncommitted changes. This ensures that the user is prompted to commit or stash work before the system modifies the filesystem, preventing potential data loss.

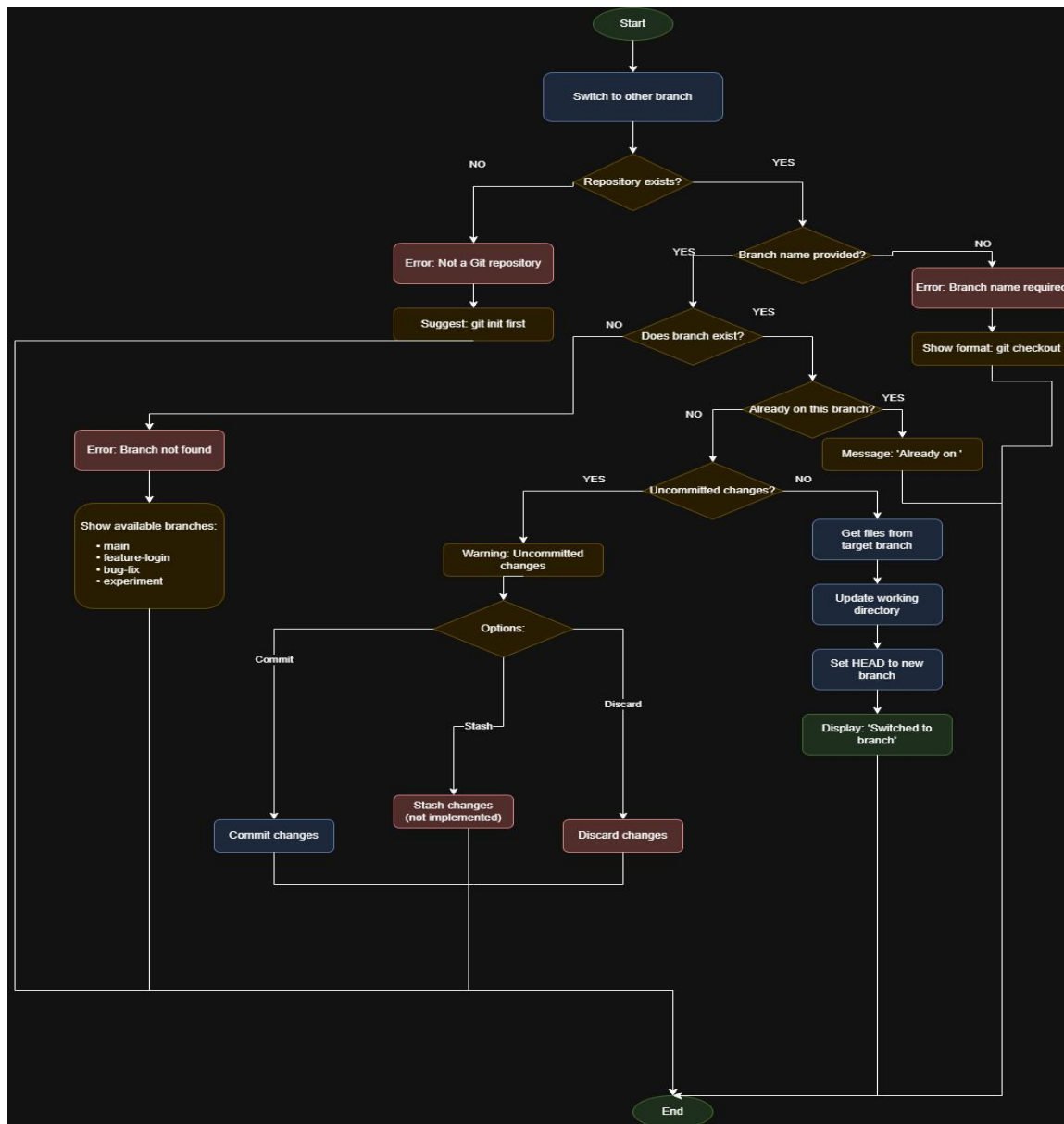


Figure 2.5: Switching Branches

- **Merging Logic:** The merge diagram models the complex process of combining branch histories. It illustrates the workflow from selecting the source branch to conflict detection. The model explicitly includes a branching path for conflict

resolution, where the user is required to manually intervene before the system proceeds to create a merge commit, thereby maintaining the structural integrity of the branch history.

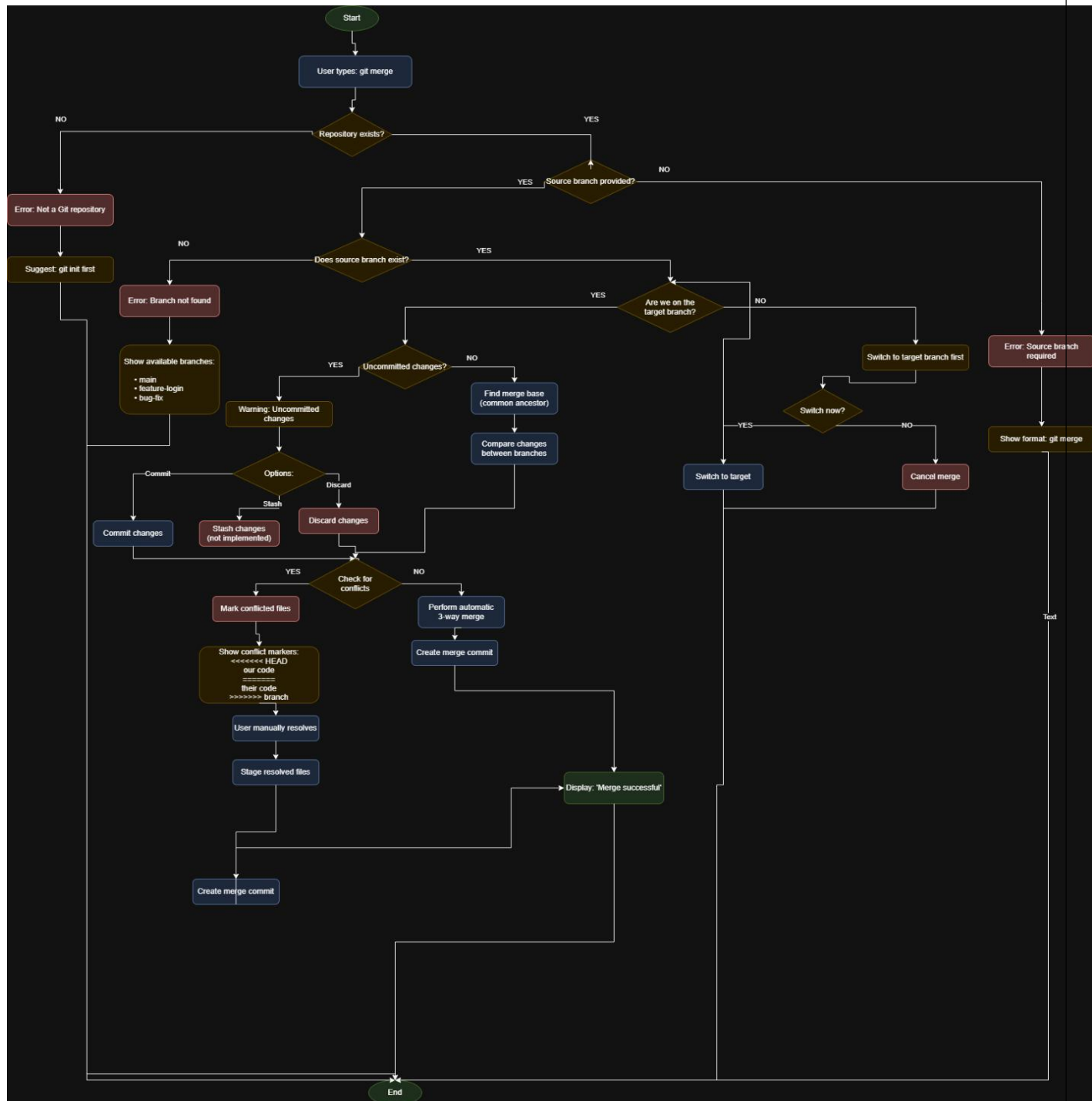


Figure 2.6: Merging Logic

4.3 Network Operations

- **Cloning Flow:** This diagram represents the synchronization of a remote repository to a local machine. The process covers the entire network lifecycle: from URL parsing and validation to the physical fetching of remote objects. It also handles edge cases, such as destination folder existence and connection failure, demonstrating the system's capability to manage external dependencies during repository setup.

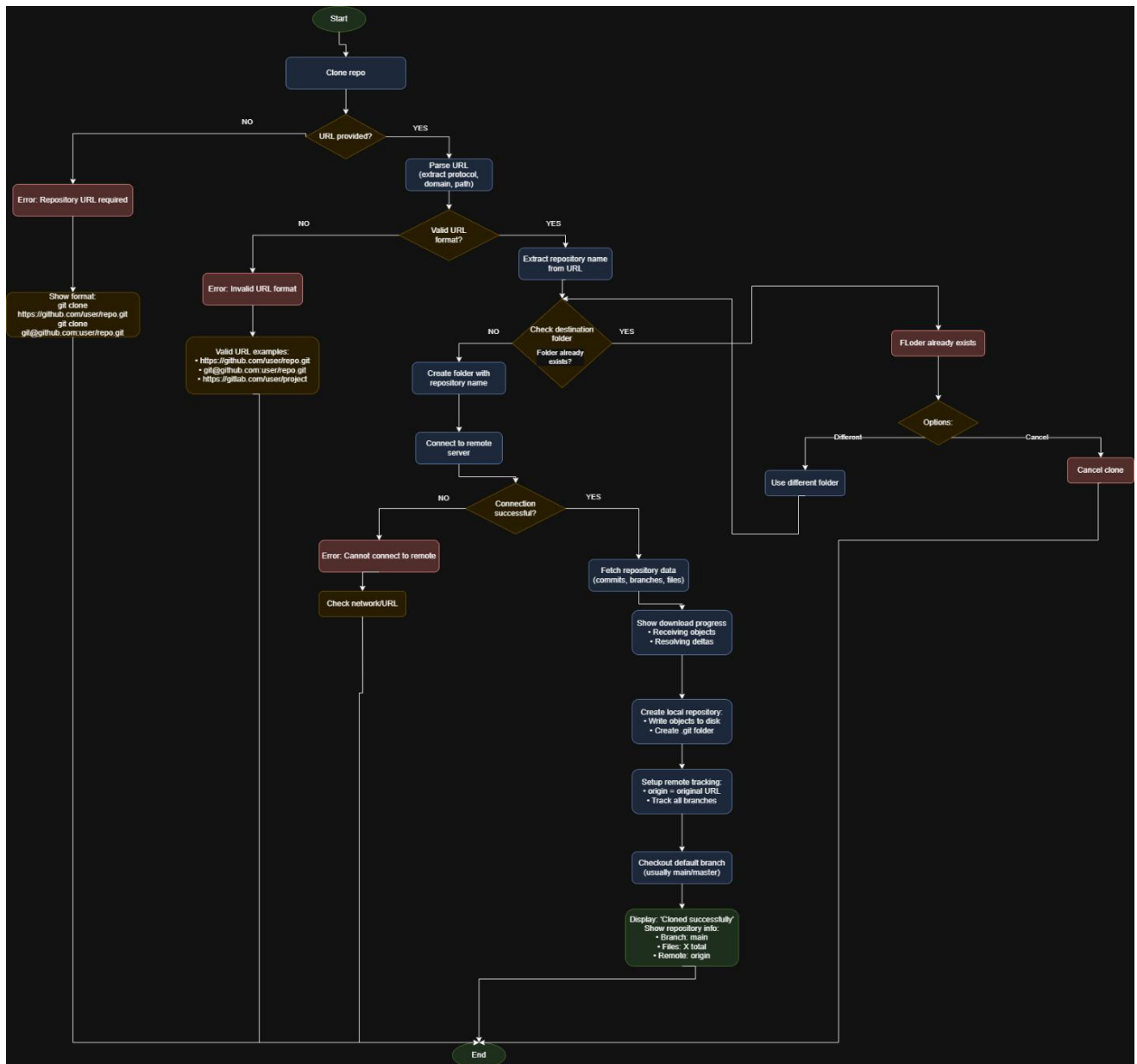


Figure 2.7: Cloning Flow

5. Structural Modeling (Class Diagram)

The class diagram serves as the static architectural blueprint of the SGVCS. It defines the software components as objects, their encapsulated data, and the governing relationships that ensure system consistency. This design adheres to the **Principle of Separation of Concerns**, ensuring that repository management, versioning logic, and network synchronization remain modular and maintainable.

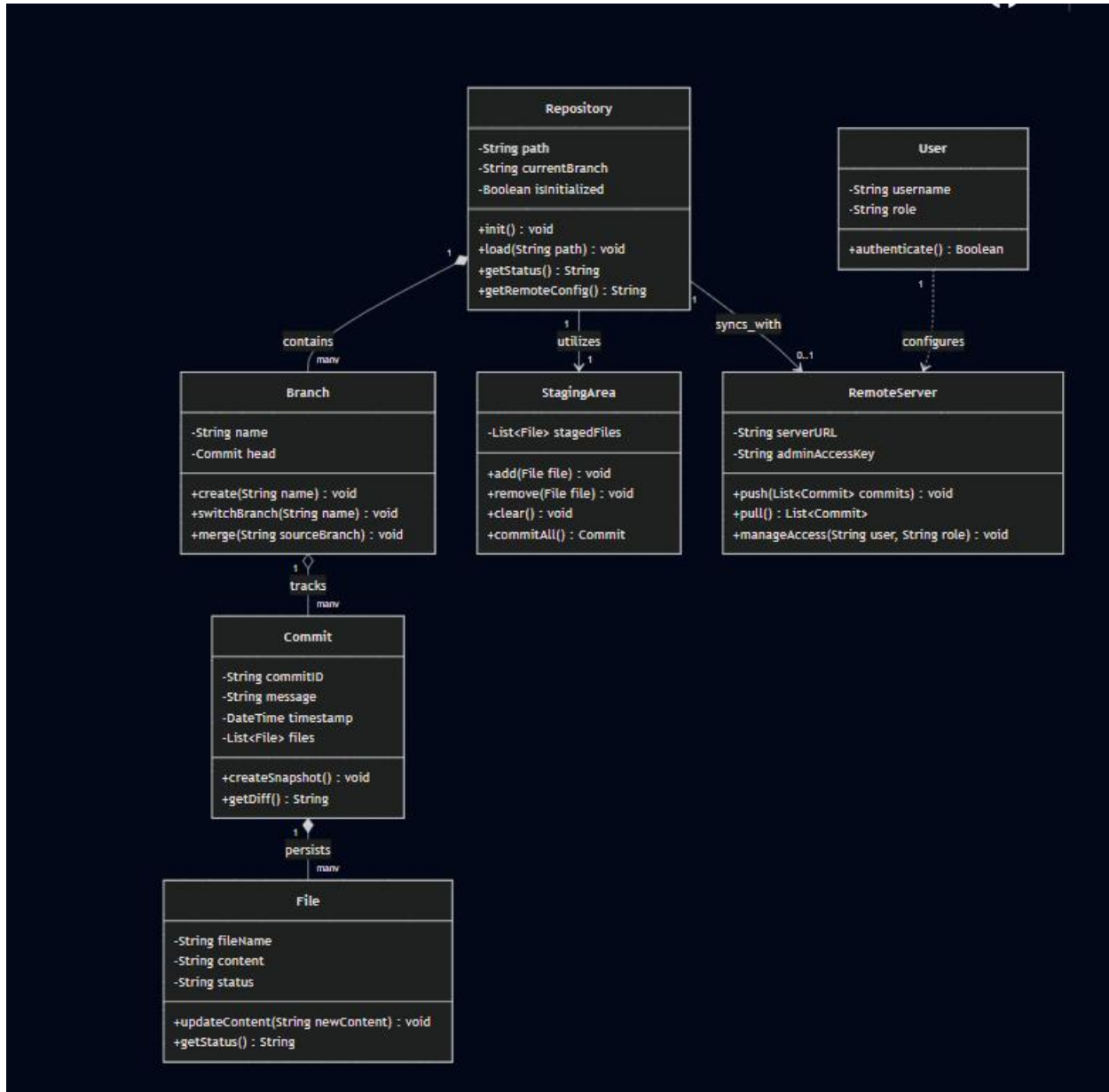


Figure 5.1: High-Fidelity Class Diagram of SGVCS

5.2 Class Definitions

- **Repository:** The root object of the system. It acts as the central controller and entry point. It manages the lifecycle of the project, including initialization and configuration of remote connections. It encapsulates the "state" of the project folder.
- **Branch:** Represents a parallel development timeline. The Branch class maintains a reference to the head commit, which serves as the pointer for current development. This class is essential for fulfilling the requirement of non-linear development.
- **Commit:** A specialized object that represents an immutable snapshot of the project. It stores metadata (timestamp, ID, message) and maintains a persistent list of the Files that existed at that specific moment in time.
- **StagingArea:** Acts as the intermediary buffer between the Repository and the permanent Commit storage. It temporarily holds a collection of Files marked for the next version update.
- **File:** The fundamental unit of data in the system. By treating a file as an object with its own status (untracked, staged, or committed), we enable granular tracking and easy status reporting.
- **RemoteServer:** Encapsulates the logic for external synchronization. It handles authentication and the transfer of commit history, keeping network-specific complexity away from local versioning logic.
- **User:** A management class that defines system roles (Developer vs. Admin), ensuring that sensitive operations like `manageAccess` are only accessible to authorized actors.

5.3 Relationships and Workflow

The system's integrity is maintained through the following defined relationships:

- **Composition (Repository → Branch):** A Repository is composed of Branches. This denotes a strong lifecycle dependency; the branches are logically tied to the existence of the repository container.
- **Aggregation (Branch → Commit):** A Branch aggregates Commits. This models the "timeline" of the project; as a developer commits changes, new objects are added to the branch's sequence.
- **Persistence (Commit → File):** A Commit persists a specific state of Files. This relationship is critical for "time-travel" (reverting code), as it allows the system to reconstruct a full project state from any past commit ID.
- **Collaborative Synchronization:** The association between Repository and RemoteServer acts as the bridge for distributed development. When a user invokes push or pull, the Repository delegates these operations to the RemoteServer class, which handles the secure data transmission.

6. Behavioral Modeling (Sequence Diagrams)

While activity diagrams illustrate the procedural flow, sequence diagrams represent the **dynamic interactions** between the system's objects. These diagrams provide a chronological view of how class methods are invoked to fulfill a specific user request.

Note: In this system, we use a "message-passing" architecture, where the `Repository` class acts as the central orchestrator that delegates tasks to specialized components like the `StagingArea`, `Commit` history, or the `RemoteServer`.

6.1 Interaction Analysis

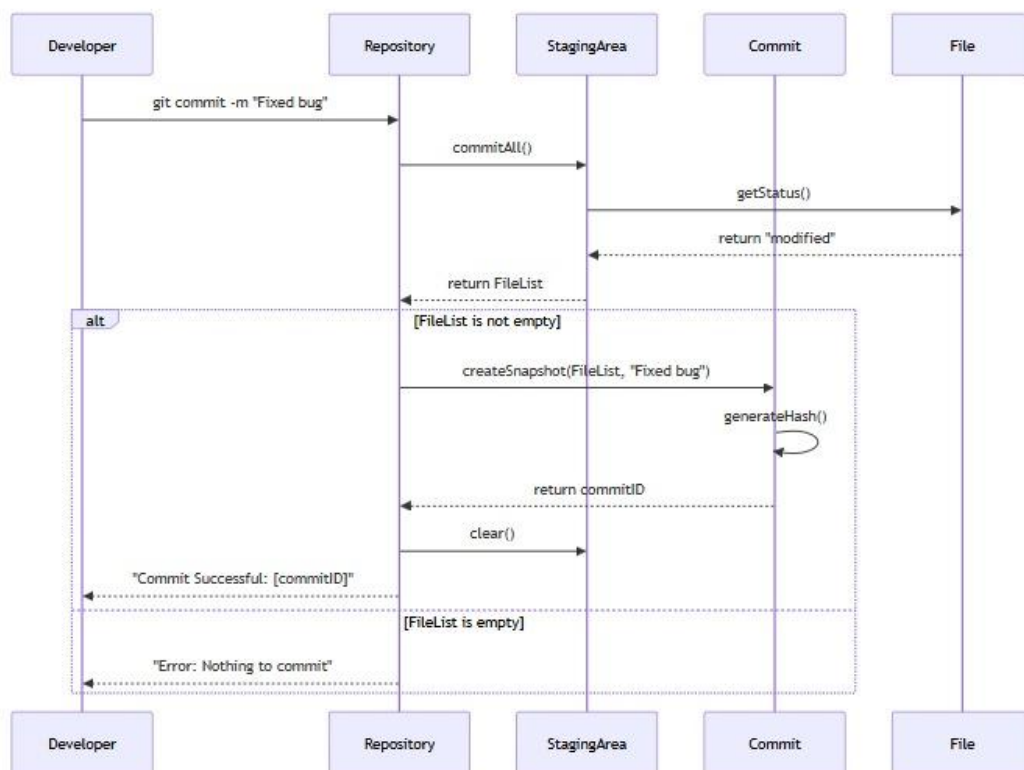


Figure 6.1: Sequence Diagram for 'Commit Changes'

System Workflow: This diagram illustrates the "snapshot" process. The `Repository` requests the current file list from the `StagingArea`. Once confirmed, it triggers the `Commit` class to generate an immutable snapshot. This demonstrates the system's ability to ensure data consistency by preventing empty commits.

- **The Story:** A developer wants to save their work ("Fixed bug").

- **How it works:** 1. The **Developer** gives the command to the **Repository**. 2. The **Repository** (the boss) asks the **StagingArea** to gather all the changed files. 3. The **StagingArea** checks each **File** to see if it's "modified." 4. Once the list is ready, the **Repository** gives the data to the **Commit** class, which acts like a camera taking a photo (snapshot). 5. The **Commit** assigns a unique ID, and the **StagingArea** is wiped clean, ready for the next task.

- **Why it matters:** It shows how the system ensures that only *valid* changes are saved, protecting the project from incomplete saves.

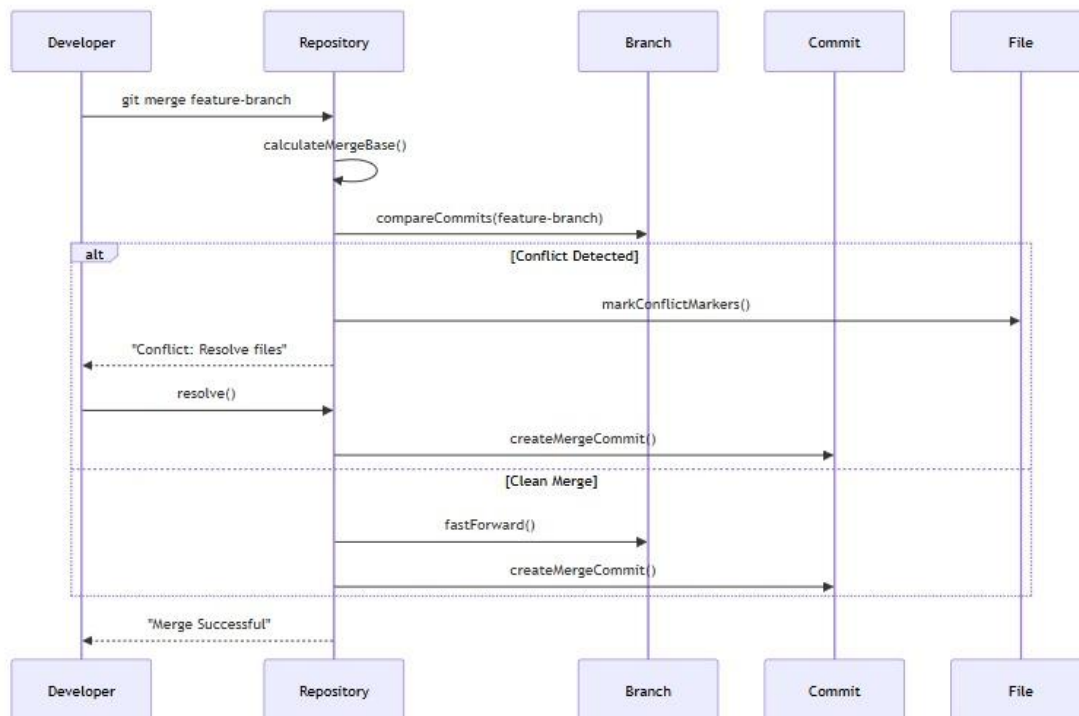


Figure 6.2: Sequence Diagram for 'Merge Branches'

System Workflow: This model demonstrates the system's "intelligence." Before performing a merge, the system executes a logic check for conflicts. If conflicts arise, it forces the system to mark the affected files, preventing data loss. This showcases the system's robustness in handling parallel development.

- **The Story:** A developer joins two parallel paths of work (e.g., merging a "feature" into "main").

- **How it works:** 1. The **Repository** calculates the "common ancestor" (where the two branches started) to see what changed. 2. It compares the **Branch** contents. 3. **The Decision (Alt Block):** If the system sees the same line changed in two places, it stops and marks a conflict in the **File**. It

sends a message to the Developer: "Resolve the conflict!" Once resolved, the system creates a "Merge Commit" to seal the deal. If everything matches, it simply updates the pointer (Fast-Forward).

- **Why it matters:** This proves the system is "smart" enough to detect when work overlaps and prevents data loss.

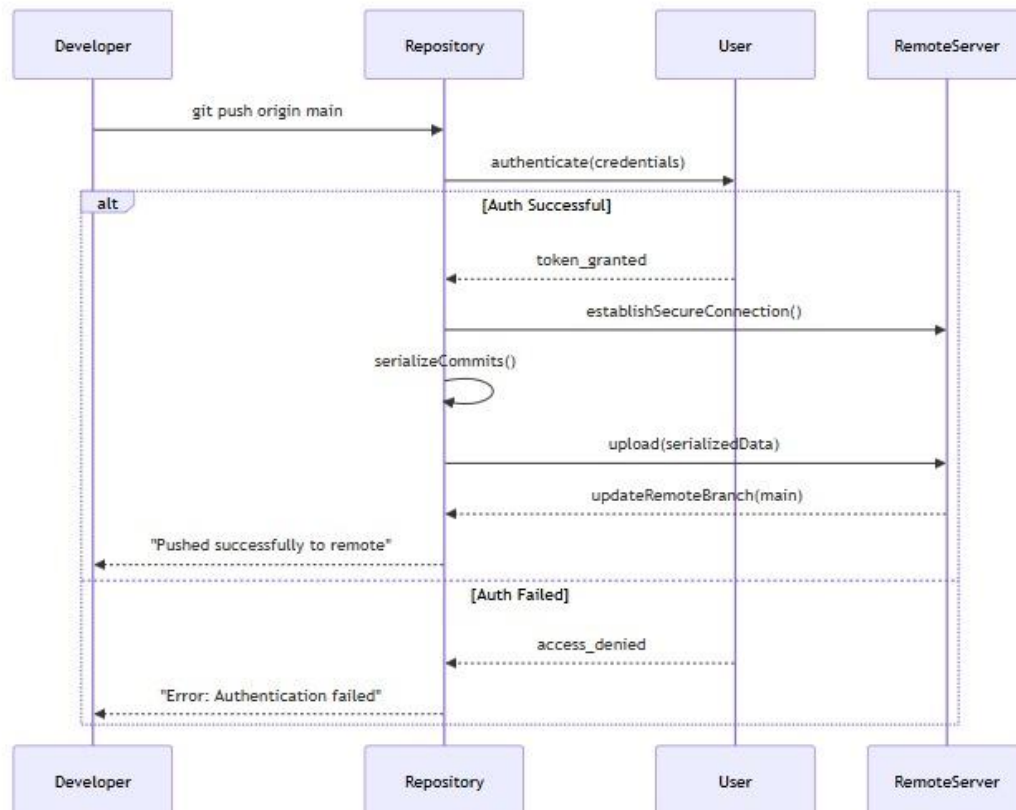


Figure 6.3: Sequence Diagram for 'Push to Remote'

System Workflow: This diagram details the distributed nature of our system. It explicitly models an `Authentication` layer. Only after the `User` is validated does the `Repository` initiate a secure communication channel with the `RemoteServer` to synchronize data.

- **The Story:** A developer sends their local history to a team server.
- **How it works:** 1. The **Developer** initiates the push. 2. Before moving any data, the **Repository** requires an **Authentication** check with the **User** class. 3. If the password/token is correct, the system opens a secure connection to the **RemoteServer**. 4. The **Repository** "serializes" (packs up) the history and uploads it. 5. If the credentials are wrong, the system hits an "Access Denied" error and shuts the process down immediately for safety.

- **Why it matters:** This establishes security. It shows that you've designed a system that prevents unauthorized people from overwriting the team's project.

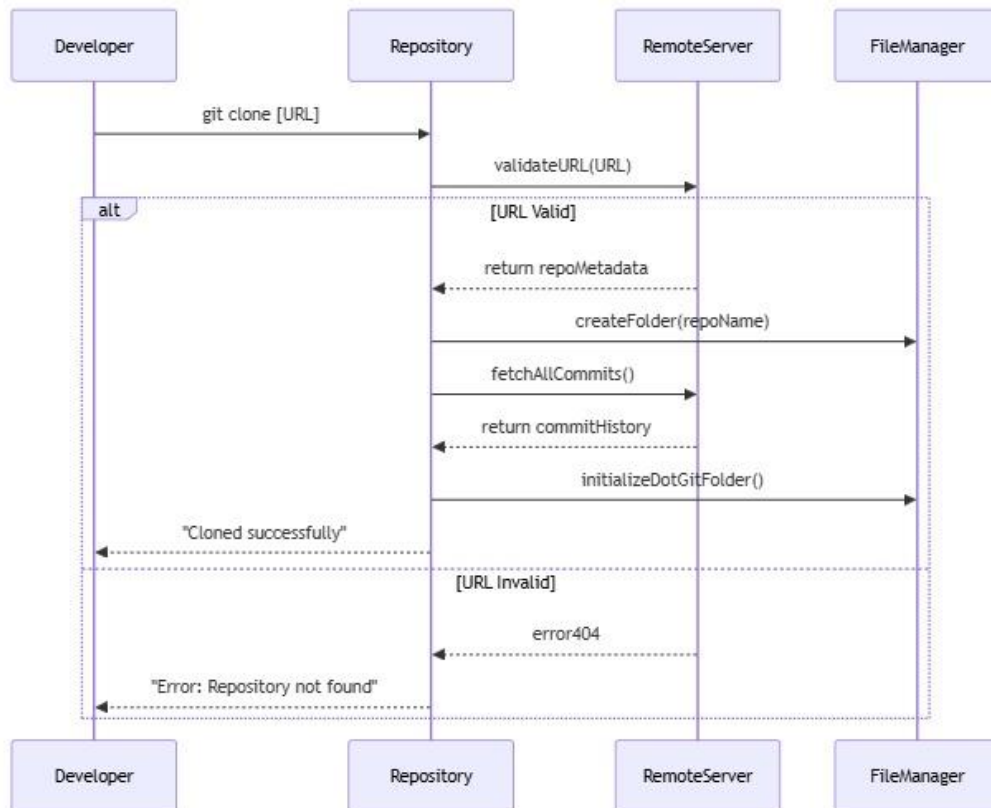


Figure 6.4: Sequence Diagram for 'Clone Repository'

System Workflow: This captures the system's "initialization" phase. It follows a validation protocol where the `RemoteServer` must confirm the existence of a repository before the `FileManager` allocates resources to the local machine, ensuring the environment is prepared before data transfer begins.

- **The Story:** A new developer gets the project for the first time.
- **How it works:** 1. The **Repository** validates the URL provided by the **Developer**. 2. If the server is found, the **Repository** asks the **RemoteServer** for all the history. 3. The **FileManager** (the worker) is then told to create a new folder and build the hidden ".git" structure. 4. Finally, the system reports "Cloned successfully." If the URL is fake (error 404), the system halts to avoid creating empty, broken folders.

- **Why it matters:** This is the system's "First Impression." It ensures that a new user gets a clean, working copy of the project.

6.3 Collaboration Analysis (System-Wide Workflow)

The Collaboration Diagram below serves as the "Master Map" of the SGVCS architecture. It illustrates the organizational links between all primary classes and defines the sequence of operations that maintain the system's state.

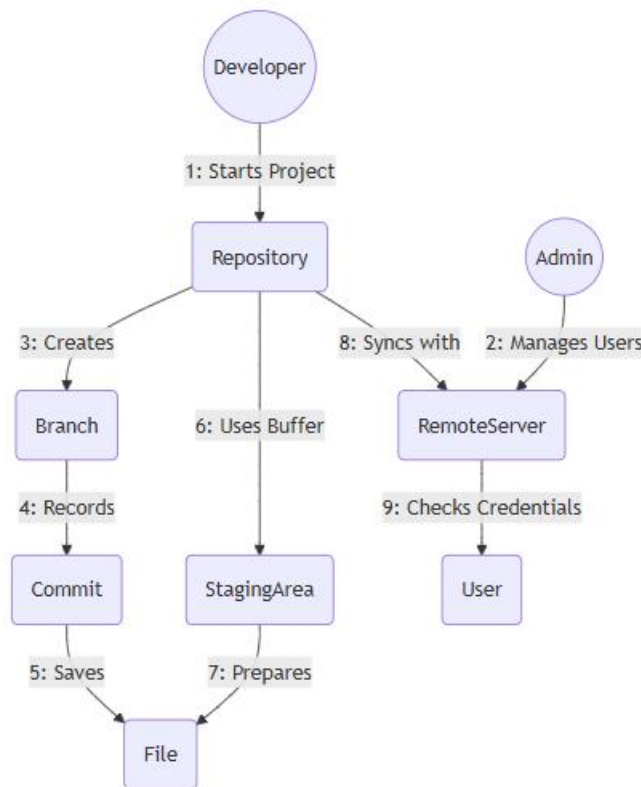


Figure 6.3: System-Wide Collaboration Map

Description:

- **Centralized Coordination:** The `Repository` serves as the primary communication hub for the entire system. Every operational request—from initial project setup (Step 1) to remote synchronization (Step 8)—is managed by this central controller, ensuring high modularity and a clear separation of concerns.
- **Operational Zones:** The diagram visualizes three functional "zones":
 - **Development Zone:** (`Developer`, `Repository`, `StagingArea`) Handles the daily creation and preparation of work.

- **Historical Zone:** (Branch, Commit, File) Manages the permanent, immutable records of the project timeline.
- **Collaboration Zone:** (RemoteServer, User) Governs the secure interaction between local environments and team servers.
- **Flow of Control:** The numbered labels (1–9) represent the logical progression of system activities. This structure proves that the system is not a collection of isolated scripts, but a well-integrated architecture where every component has a specific, defined role in the version control lifecycle.

7. Collaboration Diagram (Communication Diagram)

The Collaboration Diagram illustrates the dynamic interaction between different objects within the SGVCS system. Unlike Sequence Diagrams, which mainly focus on the order and timing of messages, this diagram emphasizes the relationships between objects and the flow of communication among system components.

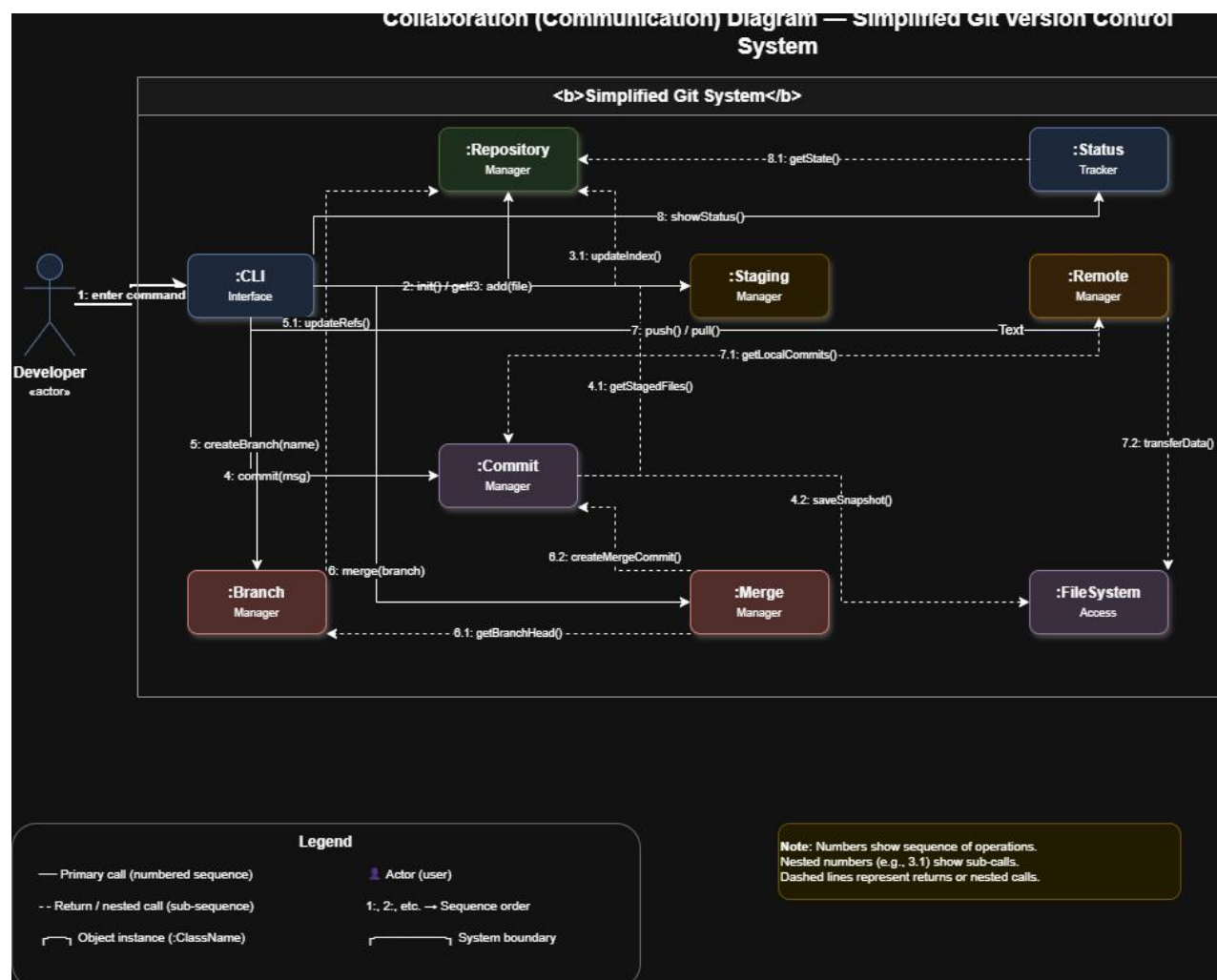


Figure 6.5: System-Wide Collaboration Map

Description

The Collaboration Diagram demonstrates how the Repository Manager coordinates different operations by receiving user commands through the CLI Interface and assigning tasks to the required system components.

Sequence Flow:

Step	Action
1	User enters a command through the CLI Interface.
2	Repository Manager invokes <code>init()</code> and <code>saveSnapshot()</code> operations in the Staging Manager.
3	Repository Manager calls <code>add(file)</code> and <code>updateIndex()</code> operations in the Staging Manager.
3.1	Staging Manager returns the list of staged files using <code>getStagedFiles()</code> .
4	Repository Manager executes <code>commit(msg)</code> to create a new commit.
4.1	Repository Manager requests staged files using <code>getStagedFiles()</code> .
4.2	Staging Manager performs <code>saveSnapshot()</code> to store the commit snapshot.
5	Repository Manager creates a new branch using <code>createBranch(name)</code> .
6	Repository Manager performs branch merging using <code>merge(branch)</code> .
6.1	Staging Manager returns the branch information through <code>getBranchHead()</code> .
6.2	Repository Manager creates a merge commit using <code>createMergeCommit()</code> .
7	Repository Manager performs remote synchronization using <code>push()</code> or <code>pull()</code> .
7.1	Staging Manager provides local commit history using <code>getLocalCommits()</code> .
7.2	Remote Manager transfers repository data using <code>transferData()</code> .

Components:

Repository Manager

Acts as the central controller responsible for coordinating repository operations.

Staging Manager

Manages file staging, commits, snapshots, and branch-related operations.

Remote Manager

Handles communication with remote repositories and synchronization tasks.

Responsible for managing file storage and disk-level operations.

- A state diagram is used to represent the condition of the system or part of the system at finite instances of time.
- It's a behavioral diagram and it represents the behavior using finite state transitions. This diagram is used in design phase of the system.
- State diagrams are also referred to as State machines and State-chart Diagrams.



This diagram illustrates the lifecycle of a file within a local Git repository. A file starts as Untracked when first created. Editing it moves it to Modified, and running `git add` transitions it to Staged. The `git commit` command saves the file to the repository history (Committed). From there, further edits return it to Modified, `git restore --staged` unstages a file, and `git rm --cached` makes it untracked again.

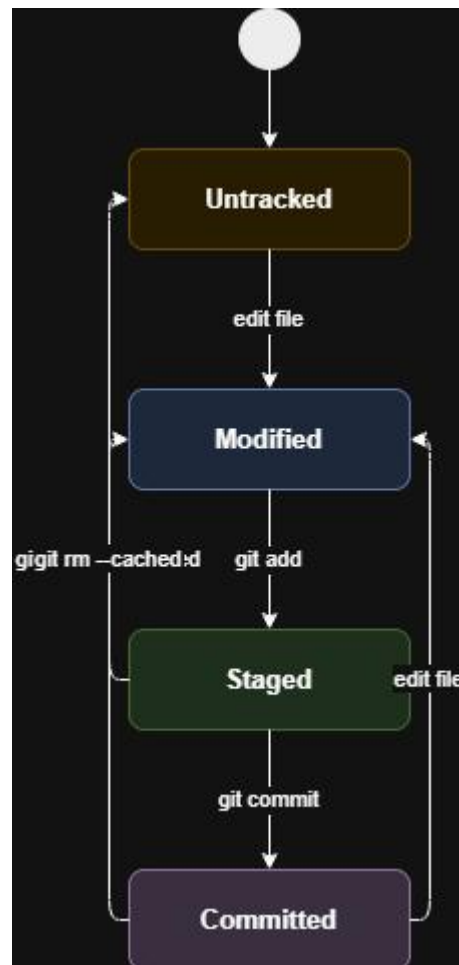


Fig 8.1: Git file state diagram

8.2. Git Repository lifecycle

This diagram maps the evolution of a Git repository from an empty folder to a fully synchronized remote repository. It begins with no `.git` folder (EmptyFolder), initializes via `git init` (Initialized), adds files (HasFiles), commits them (HasCommits), connects a remote (RemoteConnected), and finally synchronizes (Synced). The diagram also shows divergence scenarios like LocalAhead (unpushed commits) and RemoteAhead (newer commits on remote), resolved by `git push` or `git pull`.

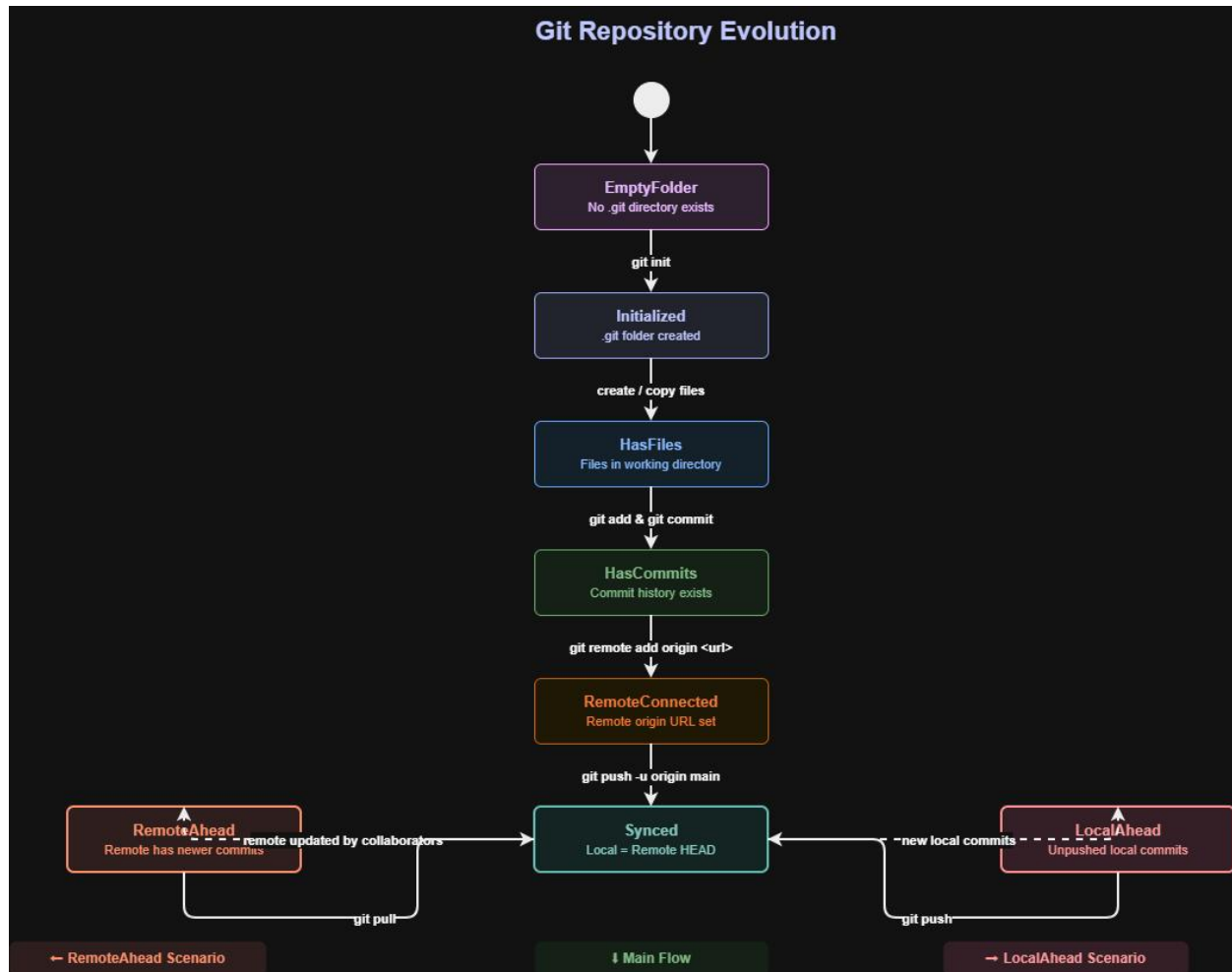


Fig 8.2: Git Repository lifecycle

8.3. Git Branch Lifecycle

This diagram shows how a Git branch is created, used, merged, and deleted. After creation via `git branch`, the branch becomes Active when checked out with `git checkout`. Commits occur while active. Merging into another branch transitions it to Merged, after which it can be safely deleted with `git branch -d`. Force deletion with `-D` removes an unmerged branch directly.

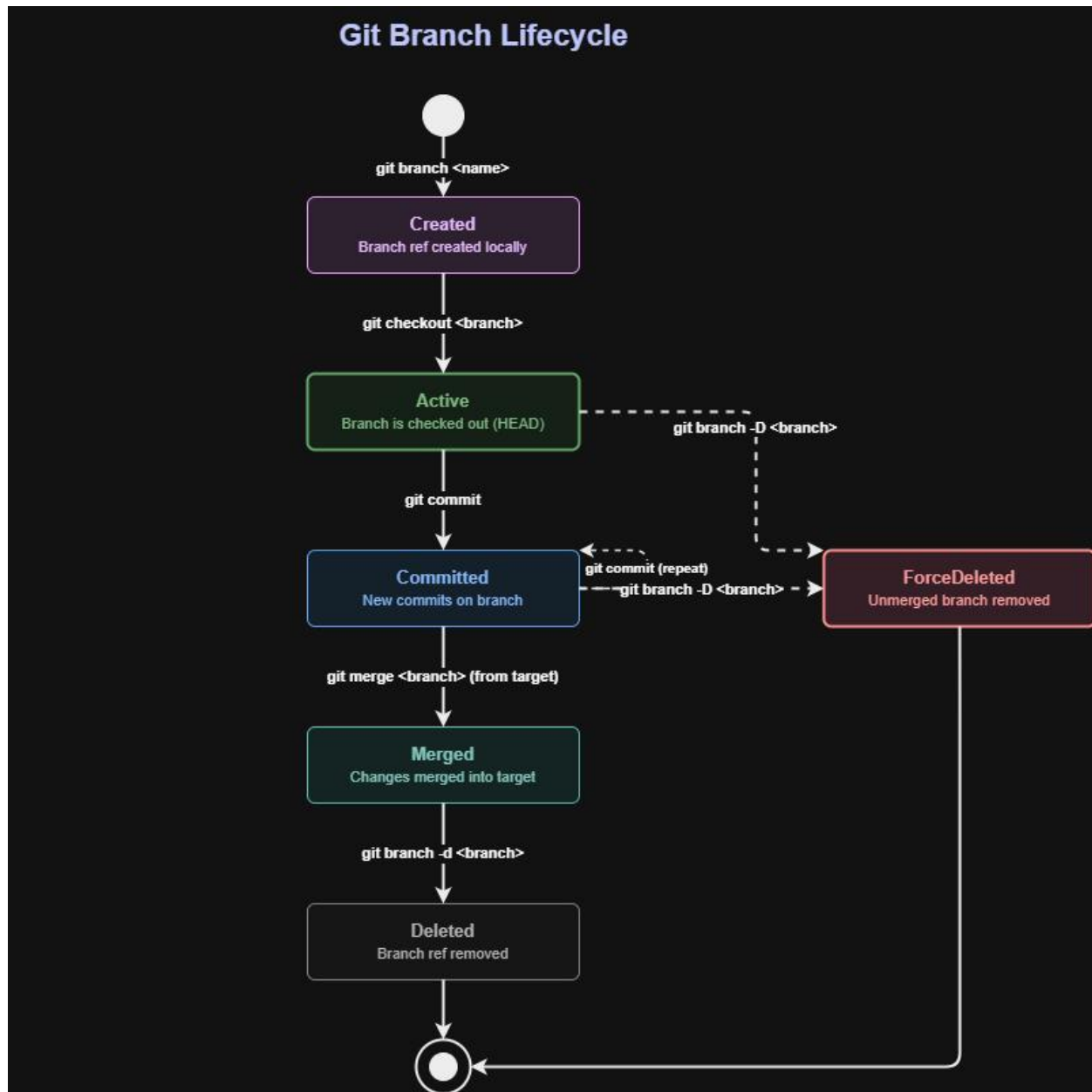


Fig 8.3: Git Branch Lifecycle

8.4. Git Merge Process

This diagram details the step-by-step merge process. Starting on the target branch (e.g., main), `git merge <source>` initiates merging. If no conflicts arise, a CleanMerge leads to an automatic commit. If conflicts are detected, the user enters ResolvingConflicts, manually edits files, stages resolved files with `git add`, and finally commits the merge. The process ends when MergeCommitted is reached.

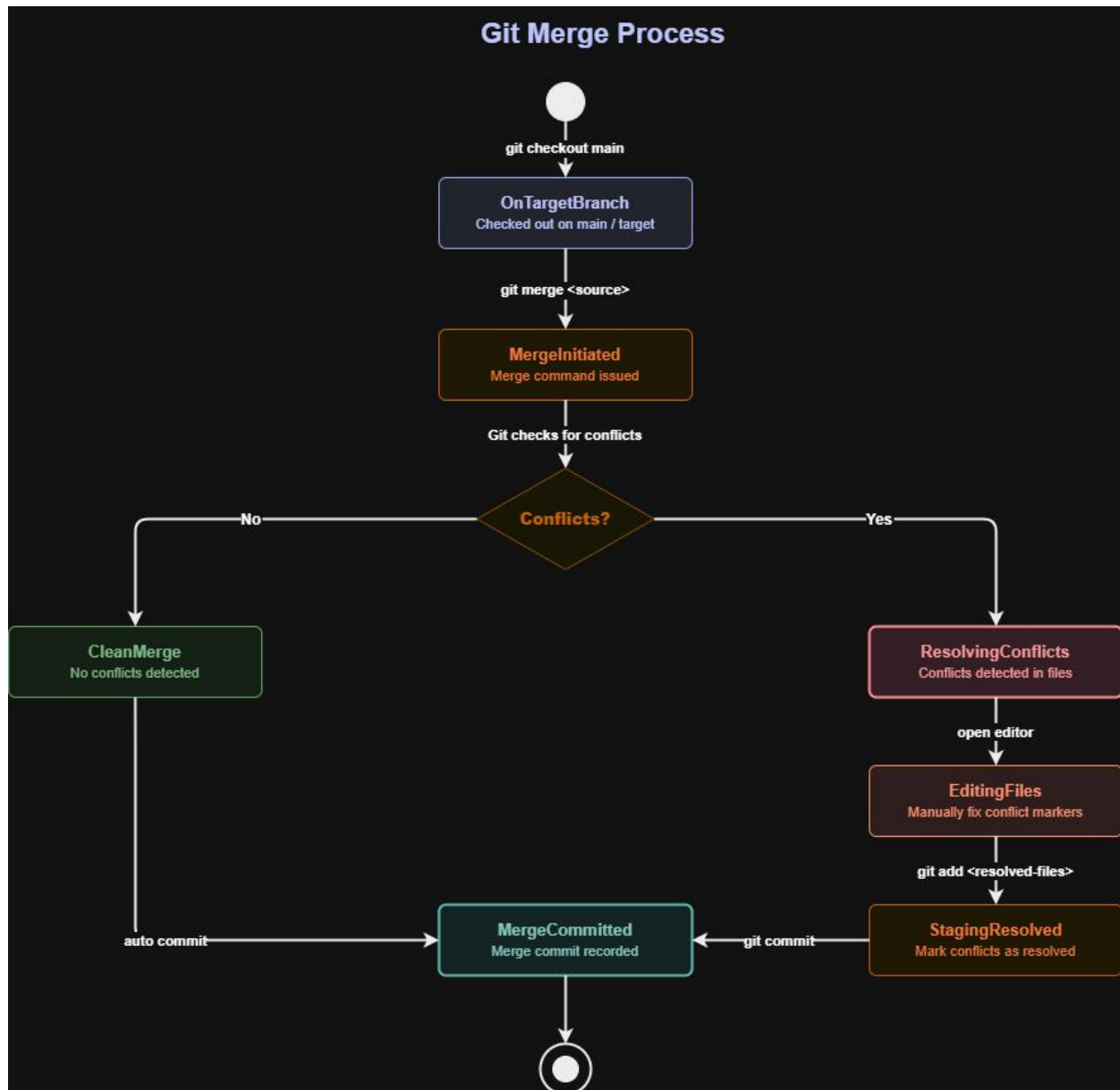


Fig 8.4. Git Merge Process

8.5. Git Remote Access Control

This diagram represents user permissions on a remote Git repository. A new user starts with NoAccess. An administrator can grant read-only access (ReadOnly), allowing cloning and pulling. Granting write access moves the user to ReadWrite, enabling pushes as well. Permissions can be revoked in reverse order, moving back to ReadOnly or directly to NoAccess. This model is typical for platforms like GitHub, GitLab, or Bitbucket with role-based access control.

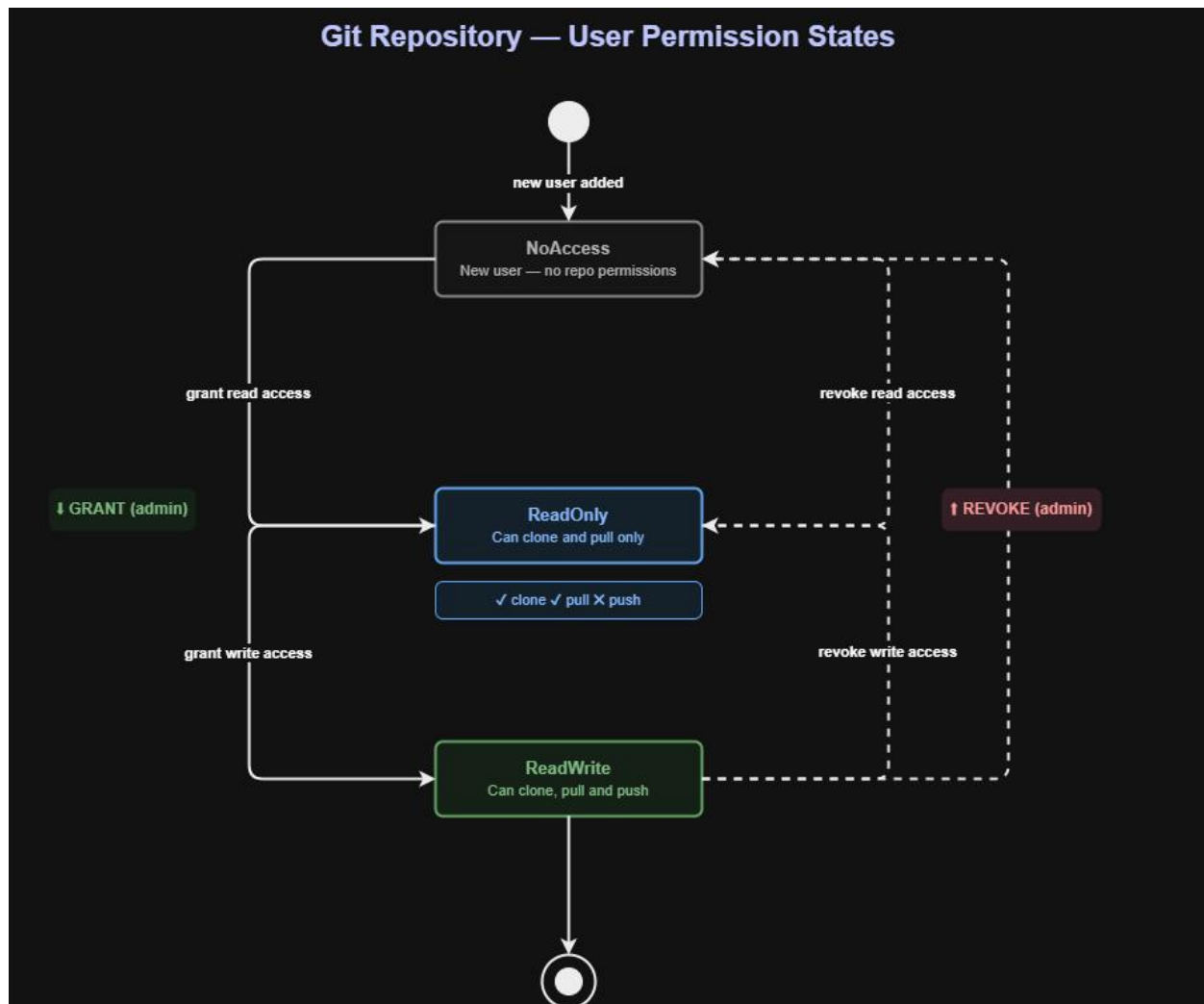


Fig 8.5: Git Remote Access Control

9. Component Diagram

Component diagram is a special kind of diagram in UML. The purpose is also different from all other diagrams discussed so far. It does not describe the functionality of the system but it describes the components used to make those functionalities.

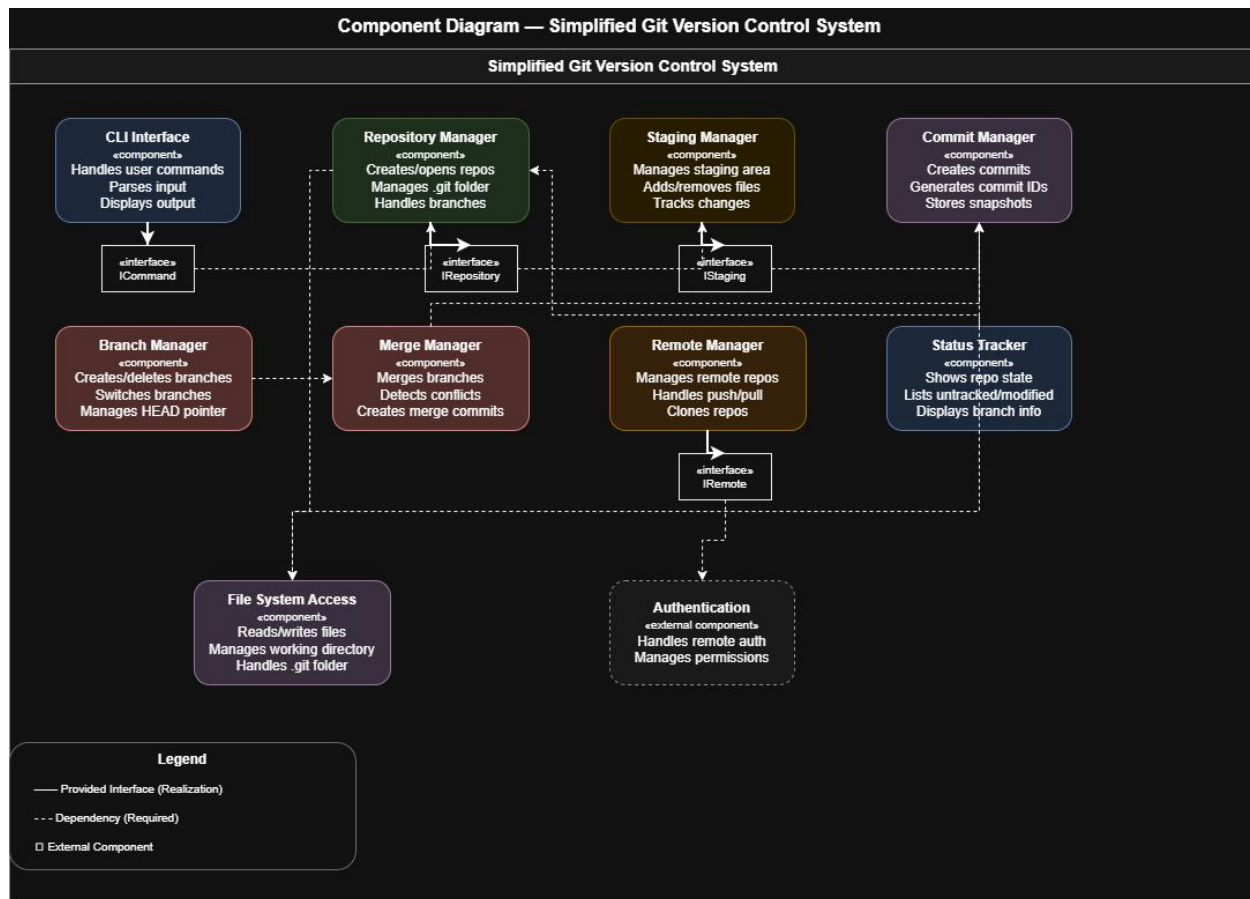


Fig 9: Component Diagram

Component Breakdown

1. CLI Interface

Purpose: Acts as the primary user interaction point

Responsibilities:

- Handles user commands (commit, push, pull, branch, etc.)
- Displays command output and status information

2. Branch Manager

Purpose: Manages all branch-related operations

Responsibilities:

- Creates and deletes branches
- Switches between branches

- Manages the HEAD pointer (current branch reference)

3. File System Access

Purpose: Handles low-level file operations

Responsibilities:

- Reads and writes files in the working directory
- Manages the .git folder structure
- Provides file I/O services to other components

4. Repository Manager

Purpose: Central coordinator for repository operations

Responsibilities:

- Creates and opens repositories
- Manages the .git folder lifecycle
- Handles branch metadata storage

5. Staging Manager

Purpose: Manages the staging area (index)

Responsibilities:

- Adds and removes files from staging
- Tracks changes before commit
- Prepares snapshots for commits

6. Commit Manager

Purpose: Handles commit creation and storage

Responsibilities:

- Creates new commits
- Generates unique commit IDs (SHA hashes)
- Stores commit snapshots to object database

7. Merge Manager

Purpose: Manages branch merging operations

Responsibilities:

- Merges branches together
- Detects and reports merge conflicts
- Creates merge commits with two parents

8. Remote Manager

Purpose: Handles remote repository interactions

Responsibilities:

- Manages remote repository connections
- Handles push and pull operations
- Clones repositories from remote sources

9. Status Tracker

Purpose Reports repository state to the user

Responsibilities:

- Shows current repository status
- Lists untracked, modified, and staged files
- Displays current branch information

10. Authentication

Purpose: Secures remote repository access

Responsibilities:

- Handles remote authentication (SSH/HTTPS)
- Manages user permissions and access control

Conclusion

The Simplified Git Version Control System (SGVCS) successfully applies core OOAD principles—encapsulation, state management, and modular design—to model essential version control mechanisms. Through clearly defined state diagrams (repository evolution, merge process, branch lifecycle, and user permissions) and a well-structured component diagram, the system demonstrates how local and remote workflows can be represented systematically. SGVCS provides a clean, educational foundation for understanding distributed version control without the complexity of production-grade systems.